The tools and techniques of R programming
Lesson-7
Data Structures in R

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

September 11, 2023

Objective of this Lesson

This lesson aims to provide with a detailed knowledge of all the data structures supported by R. These data structures equip R to handle all types of data that we face in our practical life in a simple, compact and lucid manner. It is these data structures that have enriched R for use as a very useful tool in all sorts of programming.

Prerequisite: Knowledge of a programming language will become an added advantage.

Definition of a data structure and the exclusive data structures supported by R

A data structure is a specific logical model for organizing and storing data in the main memory of a computer. R is very rich in supporting a variety of data structures which makes it a very powerful programming language for handling all types of data processing activities. These data structures are very robust and versatile. This lesson is meant for studying these data structures in detail.

R supports six different types of data structures and manipulation of data through the structures. These data structures can be classified in two primitive ways,namely,

1. Based on the nature of data they contain and
2. Based on their organization.

According to the first way of classification, the data may be either **homogeneous** i.e. comprising data of same type or **heterogeneous** i.e. consisting of data of different types.On the basis of the second way of classification, they are organized either in **one dimensional form**, or in **two dimensional form** or in **multidimensional form.**

Table showing the data structures in R

The following table shows the data structure according to their classifications.

Organization of data	Homogeneous data structure	Heterogeneous data structure
1 dimensional(1-d)	Atomic vector,factor	list
2 dimensional(2-d)	matrix	dataframe
Multidimensional(n-d)	array	

Atomic Vector: An atomic vector(simply called a vector) is a basic data structure defined to contain a set of values of same data type. It is the most elementary data structure in R. Even when we define a variable in R or point out a certain value as we have used in programming in the preceding chapters, we use a vector of single element without knowing details about it. We also used multi-element vectors using the colon operator such as v=2:9, storing the set of values from 2 to 9. We have also used the **seq()** operator to generate a sequence of values as in a for loop illustrated before. It also creates a vector.Formally,the **R function c( ) [short for combine]** is used to create a vector as illustrated below:

```
>v1=c(12,45,67)
># this hash symbol is used to write comments of the user.
>#let us now print the values of the vector
>v
[1] 12 45 67
```

Atomic Vectors-Illustration of its creation-Contd.

So,we have observed how to create a vector using the **c()** function.

Let us now create a vector using **:** operator.

>**v=2:10** # This stores the whole numbers from 2 to 10 in v

>**v** # Naming a vector implies showing its content

[1] 2 3 4 5 6 7 8 9 10 # The values stored in v are displayed.

Let us now create a a vector using the **seq()** function having the format:**seq(start-value,stop-value,step-value)**

The syntax is:**seq(from = 1, to = 1, by =** $\frac{\textbf{to - from}}{\textbf{length.out - 1}}$,

length.out = NULL, along.with = NULL, . . .)

Where **length.out** implies the number of values.

>**vec=seq(5,10,0.5)**

>**vec**

[1] 5.0 5.5 6.0 6.5 7.0 7.5 8.0 8.5 9.0 9.5 10.0

>**vv=seq(1,15,length.out=5)**

>**vv**

[1] 1.0 4.5 8.0 11.5 15.0

>**seq(1,15,length.out=6)**

[1] 1.0 3.8 6.6 9.4 12.2 15.0

Atomic Vectors-Defining Empty/Blank Vectors

Any one of the following methods can be used to define an **empty vector**:

1. Using **vector()** method
2. Using **c()** method
3. Using **numeric()** method
4. Using **rep()** method
5. Assigning **NULL** to an existing vector.

The methods are illustrated below:

```
>bv=vector()
>bv
logical(0)
>length(bv)
[1] 0
>bv=c()
>bv
NULL
>length(bv)
[1] 0
```

Atomic Vectors-Defining Empty/Blank Vectors-Contd.

```
>is.null(bv)
[1] TRUE
>bv=numeric(0)
>bv
numeric(0)
>length(bv)
[1] 0
```

Now, let us use the generic function **rep()** that replicates the values in the input data.

```
>bv=rep(NULL, 0)
>bv
NULL
>length(bv)
[1] 0
```

Now, we take the previously defined vector v.

```
>v
```

Atomic Vectors-Defining Empty/Blank Vectors-Contd.

[1] 2 3 4 5 6 7 8 9 10
>**length(v)**
[1] 9
>**v=NULL**
>**v**
NULL
>**length(v)**
[1] 0

Accessing vectors

Vector elements are accessed using **index** of vectors, which can be numeric, character or logical vectors.Using this index value, we can access or alter/change each and every individual element present in the vector. Index value starts at 1 and ends at n where n is the vector length. The other methods of accessing vectors are the following ones:

1. Using Vector
2. Using Boolean and Negative Values
3. Using Character Vectors as Index

Accessing vectors–CONTD.

Let us observe accessing vector elements **using the elements of another vector**.

```
>metros=c('Kolkata','Mumbai','Delhi','Chennai')
>vect=c(1,3)
>metros[vect]
[1] "Kolkata" "Delhi"
>metros[c(2,4)]
[1] "Mumbai" "Chennai"
```

Accessing the Vector elements **using Negative values and the Boolean values**. In R Vectors, Negative index position is used to omit those values.

```
>vect=c(-1,-3)
>metros[vect]
[1] "Mumbai" "Chennai"
>vt=c(TRUE,FALSE,FALSE,TRUE)
>metros[vt]
[1] "Kolkata" "Chennai"
```

Accessing vectors-Naming the elements

Let us now observe accessing the R Vector elements using **Character Vectors as Index values**. Here, we declare a vector with elements having names in alphabetic characters that can help us extract the vector elements using the names as shown below:

```
>v=c("W"=5,"X"=14,"Y"=23,"Z"=-32)
>v["X"]
 X
14
>v["Z"]
 Z
-32
```

Let us now try in a different way as under:

```
>vect=c(W=7,X=9,Y=11,Z=13)
>vect[W]
Error: object 'W' not found
>vect
 W X Y Z
 7 9 11 13
>vect["W"]
W
7
>vect["X"]
X
9
```

Atomic Vectors-Naming the elements-Continued

Let us now define a vector of strings:

>**No_students_passed_In_BCA_BATCHES=c("A", "B", "C")**

If we want to use the string values as the labels of the values in v1, we can use the names() function as shown below:

>**names (v1)=No_students_passed_In_BCA_BATCHES**

>**# Now let us print the vector with labels**

>**v1**

The output will be:

A B C

12 45 67

So, we can assign names to the data elements of a vector in the way illustrated above. The naming task can be performed in three other different methods also as depicted below:

Method-1: Naming while defining the vector:

>**v1=c(A=12,B=45,C=67)**

>**v1**

A B C

12 45 67

Atomic Vectors-Naming the elements-Continued

Method- 2: Naming with strings while defining the vector:

```
>v1=c("A"=12,"B"=45,"C"=67)
>v1
 A  B  C
12 45 67
```

Method-3. By creating a modified copy of a vector with use of **setNames()** function:

```
>v=setNames(5:7, c("A", "B", "C"))
>v
A  B  C
5  6  7
```

Thus, we observe that the second way of defining vectors with labels is the most straightforward and time-saving. Another important point about the names of the vector elements is that the names need not be unique. But, it will be found to be useful during character subsetting of vectors to be discussed later.

Atomic Vectors–properties

All vectors possess three basic properties as stated below:

1. **Type**, describing what it is. This can be seen by the library function: **typeof()**
2. **Length**, stating the number of elements it contains. It can be obtained by the library function **length( )**
3. **Attributes**, containing additional arbitrary metadata(facts about the object i.e. vector etc.). It can be obtained by the function: **attributes( )**

There is one more function that is often used with vectors : it is str(). It is used to print the structure of a vector with data types and sizes as shown in the figure below.

```
>str(v1)
 Named num [1:3] 12 45 67
 - attr(*, "names")= chr [1:3] "A" "B" "C"
```

Atomic Vectors–datatypes

The datatype of the data stored in a vector can checked by using the function **typeof()**. The atomic nature of a vector can be checked by using the logical function **is.atomic( )**.The following functions can also be used to check a specific datatype of an atomic vector:

- **is.numeric()**
- **is.integer()**
- **is.logical()**
- **is.character()**
- **is.double()**
- **is.complex()**
- **is.raw()**

The numeric type actually refers to data of double type. **To specify integer values in a vector, we need to put L beside a number**. **For logical vectors, we need to store TRUE or FALSE value**. If **T** or **F** is specified, **the datatype becomes double**.

The **attributes()** function is used to assign some user-defined description to a vector.

Atomic Vectors–accessing the data types

The **structure()** function can be used to define a vector along with an attribute optionally. The use of all these functions have been illustrated below:

```
>v=c(1,2,3,4)
>typeof(v)
[1] "double"
>v11=c(1L,2L,3L,4L)
>typeof(v11)
[1] "integer"
>v12=c(TRUE,FALSE,T,F)
>typeof(v12)
[1] "double"
>is.logical(v12)
[1] FALSE
>v13=c(TRUE,FALSE)
>typeof(v13)
[1] "logical"
>v14=c(T,F)
>typeof(v14)
[1] "double"
>v15=c("A","B","C")
>typeof(v15)
[1] "character"
>length(v15)
[1] 3
>attribute(v15)
Error in attribute(v15) : could not find function "attribute"
>str(v15)
 chr [1:3] "A" "B" "C"
>str(attributes(v15))
 NULL
```

Atomic Vectors–Continued

```
>attributes(v15)
NULL
>attr(v15,"This is a character vector")
NULL
>attr(v15,"attribute")="This is a character vector"
>attributes(v15)
$`attribute`
[1] "This is a character vector"
>attr(attributes(v15))
Error in attr(attributes(v15)) : either 2 or 3 arguments are required
>str(attributes(v15))
List of 1
 $ attribute: chr "This is a character vector"
>str(v1)
 chr [1:2] "RAM" "SHYAM"
>str(v)
 num [1:4] 1 2 3 4
>structure(1:5)
[1] 1 2 3 4 5
.atomic(v)
[1] TRUE
>v16=c(1.24,4.45,9.3456)
>typeof(v16)
[1] "double"
>v17=c(2+3i,4+5i)
>typeof(v17)
[1] "complex"
>vv=c(as.raw(20),as.raw(36))
>typeof(vv)
[1] "raw"
>vv
[1] 14 24
```

Error as there is no ". atomic()" logical function instead of there is logical function known as "is.atomic()"

Atomic Vectors–Continued

We can define a vector with values containing further vectors of same type which can be done in a recursive manner.Since, **vectors do not possess any recursive property**, all such references of vectors will form a single set of values for the starting vector. This is illustrated below:

```
>vvv=c(1,2,3,c(4,5,6,c(7,8,9,c(10,11,12,13))))
>vvv
 [1] 1 2 3 4 5 6 7 8 9 10 11 12 13
```

Coercion of the elements of a vector

In programming languages, the term **coercion** means implicit conversion of the data type of a value. As by definition of a vector, we know that it will always contain a sequence of values of the same data type. So, if a user tries to store values of different data types in a vector, it will convert the values into a single data type automatically by upgrading all **logical to numeric**,**numeric to character** and **integer to numeric** in case of such necessity. This is illustrated below:

```
>s=c(2,3,6,"a")
>s
[1] "2" "3" "6" "a"
>y=c("a","b",True)
Error: object 'True' not found  This is because TRUE is misspelled in terms of case of the letters.
>y=c("a","b",TRUE)
>y
[1] "a" "b" "TRUE"
>f=c(4.0,6.11,7.56,8)
>f
[1] 4.00 6.11 7.56 8.00
```

Atomic Vectors–Continued

We may combine two vectors also. Coercion of the vector elements occurs in case the data types of the vectors differ. This is illustrated below:

```
>v1=c(1,2,3)
>v2=c('A' ,'B','C')
>z=c(v1,v2)
>z
[1] "1" "2" "3" "A" "B" "C"
```

Creation of subsets of a vector

We can take a subset of a vector and store it in a new vector. Such breaking up of a vector into chosen parts resulting in the creation of new vectors is referred to as vector subsetting. The following R codes illustrate the idea:

```
>x=5:9 # Observe how the colon(:) operator is also used to create a vector
>x
[1] 5 6 7 8 9
>v=x[c(1,3,5)]  # Vector subsetting to create a new vector with the first, third and the fifth element.
>v
[1] 5 7 9
```

Atomic Vectors–Continued–Creation of subsets of a vector

There are **five ways to subset a vector** as illustrated below:
Let us define a vector as under:
vec=c(3.1,4.2,5.3,7,4)

- **Positive integers return elements at the specified positions** as illustrated below:
  >**vec[c(2,4)]**
  [1] 4.2 7.0 # Elements of the second and the 4th positions are shown
  >**vec[c(3,2)]**
  [1] 5.3 4.2 # Elements of the third and the second positions are shown, order being immaterial
  >**vec[order(vec)]**
  [1] 3.1 4.0 4.2 5.3 7.0 # Shows the elements in a sorted form
  >**vec[c(1,1)]**
  [1] 3.1 3.1 # Duplicate values are shown if position numbers are duplicated
  >**vec[c(2.3,5.9)]**
  [1] 4.2 4.0 # Real numbers used to point out positions are truncated to integers silently

Vectors–Creation of subsets of a vector-Contd.

- **Negative integers omit elements at the specified positions**
  This is illustrated below:
  >vec[-c(2,1)]
  [1] 5.3 7.0 4.0 # Values of first and second locations are omitted
  >vec[c(-3,5)] # Mixing of positive and negative integers are disallowed resulting in error.
  Error in vec[c(-3, 5)] : only 0's may be mixed with negative subscripts
  >vec[c(-2,-3)] # positions to be omitted can also be specified
  [1] 3.1 7.0 4.0
  >vec[c(1,0)] # The subscript of a vector can never be zero; so it is just ignored
  . [1] 3.1

- **Logical vectors select elements where the corresponding logical value is TRUE. This is probably the most useful type of subsetting because we write the expression that creates the logical vector.**
  This is illustrated below:
  >vec[c(TRUE,FALSE,TRUE,FALSE,TRUE)]
  [1] 3.1 5.3 4.0 #Values of the positions indicated by TRUE are only returned
  >vec[c(TRUE,TRUE,FALSE,FALSE,TRUE,TRUE)] # If non-existent position is indicated, it returns NA
  [1] 3.1 4.2 4.0 NA
  >vec[c(TRUE)] # if the number of logical values mentioned is less than the number of elements in the vector,then the value is recycled for for all the available positions in the vector.
  [1] 3.1 4.2 5.3 7.0 4.0
  >vec[c(TRUE,FALSE)] # same reason as the preceding one.
  [1] 3.1 5.3 4.0
  >vec[c(1,NA,4)]
  [1] 3.1 NA 7.0 # subscript NA indicates that the position is not available,so,the output is also NA to point out missing value.
  >vec[]
  [1] 3.1 4.2 5.3 7.0 4.0
  >vec[0]
  numeric(0)

Vectors–Creation of subsets of a vector-Contd.

- **If the vector is named,we can also use the names as the subscripts.** This is illustrated below:

```
>vec=setNames(vec,LETTERS[1:5]) # Assigning names with the first five letters of the English alphabet
>vec # Printing the vector with the assigned names
 A B C D E
3.1 4.2 5.3 7.0 4.0
>vec[c("A","C","E")] # Using names as subscripts
 A C E
3.1 5.3 4.0
[c("A","A")] # The names can be repeated to get the repeated value as obtained with integers
 A A
3.1 3.1
```

Correct code is vec[c("A","A")]

```
>vect=c(RAM=5,RAHIM=7) # A new vector with new names of the elements
>vect[c("RA","RA")] # The names used as subscripts must have exact matches with the assigned names
<NA><NA>
NA NA #This happens because the names do not match exactly.
```

Subsetting and Assignment in Atomic Vectors

All subsetting operators can be combined with assignment operation to modify selected values of the input vector as illustrated in the figure below:

```
vec=c(2,3.5,5,6,7.1,8)
>vec[c(2,3)]=c(3.9,4.8) # Two new values are being assigned
>vec
[1] 2.0 3.9 4.8 6.0 7.1 8.0
>vec[-c(1,2)]=8:5 #Number of values to be inserted must match with the number of vector elements being referenced
>vec
[1] 2.0 3.9 8.0 7.0 6.0 5.0
>vec[c(2,NA)]=c(2,3) # NA is not allowed with integer subsetting and hence results into an error
Error in vec[c(2, NA)] = c(2, 3) :
NAs are not allowed in subscripted assignments
>cvec=c('M','S','D','W','D','W','S','S','M','M','W','S') # A character vector is defined
>lookup=c(M='Married',S='Single',D='Divorced',W='Widowed') # A character vector with labels is defined
>lookup[cvec] # The elements of cvec are being used for subsetting
   M          S          D          W          D          W          S
"Married"  "Single"  "Divorced"  "Widowed"  "Divorced"  "Widowed"  "Single"
   S          M          M          W          S
"Single"  "Married"  "Married"  "Widowed"  "Single"
>unname(lookup[cvec]) # This is to remove the names from the display with the function unname()
 [1] "Married" "Single" "Divorced" "Widowed" "Divorced" "Widowed"
 [7] "Single" "Single" "Married" "Married" "Widowed" "Single"
>append(cvec,'U') # This is to add a new element at the end of the vector.
[1] "M" "S" "D" "W" "D" "W" "S" "S" "M" "M" "W" "S" "U"
>cvec=c(cvec[1:13],'U','U') # This is another way of adding elements to a vector;1:13 implies existing number of elements
>lookup=c(lookup[1:4],U='Undefined') # Adding another value with a new label
>lookup
   M          S          D          W          U
"Married"  "Single"  "Divorced"  "Widowed"  "Undefined"
```

```
>lookup[cvec] # Having a lookup again
  M          S          D          W          D          W
 "Married" "Single" "Divorced" "Widowed" "Divorced" "Widowed"
  S          S          M          M          W          S
 "Single" "Single"  "Married"  "Married" "Widowed" "Single"
<NA>     U                    U
 NA     "Undefined"          "Undefined"
>unname(lookup[cvec])
 [1] "Married" "Single" "Divorced" "Widowed" "Divorced" "Wid-
owed"
[7] "Single" "Single" "Married" "Married" "Widowed" "Single"
[13] NA "Undefined" "Undefined"
>order(cvec)
 [1] 3 5 1 9 10 2 7 8 12 14 15 4 6 11 13 # Here the integer values
correspond to the values in alphabetic order
```

Insertion and Deletion of elements to/from a vector

We may insert or delete elements in a vector. The following examples illustrate the techniques which are self explanatory.

Insertion and Deletion of elements to/from a vector

```
>v3=c(1,2,3)
>v3=c(v3[1:2]) # This is to delete the third element and reassign the newly created vector to the previous vector location.
>v3
[1] 1 2
>v3=c(v3[1:2],7,v3[3]) # This is to insert the element 7 in the vector v3 and by mistake, we mention the same location resulting in inserting a fourth element without value.
>v3
[1] 1 2 7 NA
>v3=c(v3[1],9,v3[2]) # This is to express that the new value 9 is to be inserted after the element v[1] and at the location v[2].
>v3
[1] 1 9 2
>v3=c(v3[1:3],165) # This is to insert the value 165 after the first 3 elements of the vector.
>v3
[1] 1 9 2 165
```

Appending to a vector

The **append(v, element)** can be used to add an element at the end of a vector.

Vector manipulation

Vector arithmetic

All the arithmetic operations shown in the section 1.3.1 can also be applied on vectors to work on each of the elements. So, we can perform addition, subtraction, multiplication, division, modulo operation and exponentiation operations on each of the vector elements with a scalar value or with the corresponding elements of another vector provided that the vector elements are of numeric or integer data type. **Two vectors of same length can be added, subtracted, multiplied or divided to generate a resulting vector with the output**.

Some of these operations have been illustrated below:

```
>q=c(1,2,3)
>q
[1] 1 2 3
>r=c(5,6,7)
>q*r
[1] 5 12 21
>q-r
[1] -4 -4 -4
>q^r
[1] 1 64 2187
>r%%q
[1] 0 0 1
>q*5
[1] 5 10 15
```

Vector recycling and sorting

If two vectors of different lengths are used to perform some vector arithmetic, then, it is performed by R by recycling the elements of the shorter vector to equalize the length. This illustrated in the figure below:

```
>v1=c(1,2,3,4)
>v2=c(1,2)
>v=v1+v2
>v
[1] 2 4 4 6
>v2
[1] 1 2
```

Thus, it is seen that, v2 becomes:v2(1,2,1,2) internally before the addition process.

Vector sorting

We can also sort the elements within a vector either in ascending (increasing) order or in descending (decreasing) order, by using the **sort()** function. This is illustrated below:

```
>v
[1] 2 4 4 6
>t=sort(v,decreasing=TRUE)
>t
[1] 6 4 4 2
>x=c(2,4,-3,-2,0,7,9,1)
>y=sort(x)
>y
[1] -3 -2 0 1 2 4 7 9
```

Sorting of character vectors and Defining a blank vector

The character vectors can also be sorted similarly as illustrated in the figure below:

```
>l=c('Ram','Shyam','Jadu','Madhu')
>m=sort(l)
>l
[1] "Ram" "Shyam" "Jadu" "Madhu"
>m
[1] "Jadu" "Madhu" "Ram" "Shyam"
```

Defining a blank vector: Sometimes, we need to define a vector for reserving spaces for a large number of values which will become available at a later stage. This can be done with the **vector( ) function** with the following syntax:

User-defined-vector-name=vector(mode='datype of the values to be inserted',length=value/variable)

For example, if we write: **primes=vector(mode='integer',length=100)**

Then, the vector primes becomes a vector of length 100 to hold integers.Such vectors are very useful in programming as we shall see later.

The R any() and all() functions and the use of %in% operator

The R **any( )** function is used to check one or more vectors to see if a certain given condition is true or false for any of the elements in the vectors. The **all()** function checks similarly except that it checks the condition for all the values and returns TRUE or FALSE. The following figure illustrates the idea:

```
x=c(2,4,-3,-2,0,7,9,1)
>any(x>7)
[1] TRUE
>all(x>9)
[1] FALSE
```

This operator is used to compare the values of two vectors to determine whether the elements of the shorter vector exists in the longer vector. The comparison is done elementwise and returns the logical values TRUE or FALSE for each of the elements in the common positions of the two vectors. Its use has been illustrated in the figure below:

```
>x=c(1:20)
>y=c(15:40)
>x%in%y
 [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[13] FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

Manipulation of strings in a vector

There are many string manipulation functions in R. We discuss here some of the important ones.

grep(): This function is basically used for searching a pattern in a vector of strings. The syntax is:

grep(pattern, v),

where, **'pattern'** is any string to be matched with any part of the strings in the vector **'v'** of strings and returns the position numbers of the strings in **'v'** where the pattern is found.For example,

```
>v=c("computer science","computer applications","biological science","phys
science","philosophy")
>grep("science",v)
[1] 1 3 4
>grep("Engineering",v)
integer(0)
```

It may be observed that in the first use of grep(), the pattern 'science' is found to be present in the strings of the vector v at the position number1:1,3,4; and hence the output. On the contrary, in the second example, the pattern "Engineering" is found nowhere in the vector v; that is why, R returns integer(0).

grep() in finding regular expressions

There is another use of the function and it is find out a regular expression. We know that a regular expression is a kind wild card that shortly represents a broad class of strings. In this case,the syntax is:

grep("[string]", x), where x is either a string or a vector.

The function checks whether any of the characters in the [string] matches with occurrences of the strings in the vectors and if a match is found, it returns the element number of the vector. Its use has been illustrated below:

```
>grep("[aeiouAEIOU]","Education")
[1] 1
>grep("[pu]",v)
[1] 1 2 4 5
```

Here, the vector v is as defined above.

In a wildcard, a period (.) represents any single character. So, if a regular expression contains a period preceded and followed by some character, it implies any substring in the elements of a vector preceded and followed by the same characters with any character in place of the period. This is illustrated below:

```
>grep("p.t",c("computer","calculator"))
[1] 1
>grep("p.t",v)
[1] 1 2
```

grep() in finding regular expressions –Contd. and nchar() explained

If the period itself is searched then it must be preceded by the escape character backslash; but the backslash(\) itself is to be escaped; so, double backslash(\\) will be required for such a search. This is illustrated below:

>**grep(".",c("compu.book","tompu.took","calcu"))**

[1] 1 2 3 # This is a wrong output. It has occurred because the period was not escaped, so, it interprets '.' As a wildcard character or meta character. It should have been written as follows to search for the period:

grep("\\.",c("compu.book","tompu.took","calcu"))

[1] 1 2

The **nchar()**.This function returns the number of characters in a given string. If the string represents a vector of strings, then the function returns an array of integers corresponding to the lengths of each of the strings in the vector. The syntax of the function is:

nchar(x)

where, x is either a string or a vector of strings.The following examples illustrates the use of the function:

>**nchar("abcdef")**

[1] 6

>**nchar(v)**

[1] 16 21 18 16 10

Here, **v** is as defined in the preceding illustration.

Illustrative Uses of strsplit() Contd. and the regexpr()

```
[1] "comput" " science"
[[2]]
[1] "comput" " applications"
[[3]]
[1] "biological science"
[[4]]
[1] "physical science"
[[5]]
[1] "philosophy"
y="ter"
>strsplit(v,y)
[[1]]
[1] "compu" " science"
[[2]]
[1] "compu" " applications"
[[3]]
[1] "biological science"
[[4]]
[1] "physical science"
[[5]]
[1] "philosophy"
```

regexpr(): The syntax of the function is:

regexpr(string-to-be-matched/variable–containing-the–string, String/variable/vector –of-strings where matching is to-be- found)

The function returns a number of values as illustrated below.

Illustrative applications of regexpr()

The first value indicates the starting position number where a match has been located, the length of the match,type of the value and whether the **"useBytes"** parameter is TRUE or FALSE. Some applications of the function have been demonstrated below:

```
regexpr(y,v)
[1] 6 6 -1 -1 -1
attr(,"match.length")
[1] 3 3 -1 -1 -1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
regexpr("ter","computer-tomputer")
[1] 6
attr(,"match.length")
[1] 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
 regexpr("ua","equater")
[1] 3
attr(,"match.length")
[1] 2
 z=regexpr(y,v)
 z
[1] 6 6 -1 -1 -1
```

The gregexpr() in R

gregexpr(): The syntax of the function is as stated below:

gregexpr(string-to-be-matched/variable–containing-the–string, String/variable/vector –of-strings where matching is to-be- found)

This function returns the same form of output as the regexpr() except that it returns the result for all the occurrences of the string. This is illustrated below with the variable y and the vector v.

```
>gregexpr(y,v)
[[1]]
[1] 6
attr(,"match.length")
[1] 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
[[2]]
[1] 6
attr(,"match.length")
[1] 3
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
[[3]]
[1] -1
attr(,"match.length")
[1] -1
attr(,"index.type")
```

The output of gregexpr() contd.

```
[1] "chars"
attr(,"useBytes")
[1] TRUE
[[4]]
[1] -1
attr(,"match.length")
[1] -1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
[[5]]
[1] -1
attr(,"match.length")
[1] -1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

An illustrative application of vectors

Example: It is required to develop an R script to count the number of vowels and consonants from a given alphabetic string.
Task Analysis: A given alphabetic string is stored in an atomic vector as a whole. So it is required to store it in a vector of size equal to the number of characters in the string. We use here the library function **nchar()** to find out the number of characters in the given string and then separate the characters one by one by using another library function s**ubstr()** having the syntax:**substr(vector, start-position,stop-position)** to store the characters in a vector of size **nchar()** that has been defined earlier. Now, we can make the comparison with all the vowels stored separately in a vector named vowels. The rest of the thing is very easy to understand.The code of the developed script is shown below:

The R script determining the number of vowels and consonants in an alphabetic string

```
string=vector(mode="character",length=20)
s=readline(prompt="Enter any string")
n=nchar(s)
for(i in 1:n){
string[i]=substr(s,i,i)}
vowels=c('a','e','i','o','u','A','E','I','O','U')
vcnt=0;ccnt=0
for(i in 1:n){
for(j in 1:10){
if(string[i]==vowels[j]){
vcnt=vcnt+1}}}
ccnt=n-vcnt
cat("The number of vowels in the given string is: ",vcnt,"and the number consonents is: ",ccnt)
```

The Concatenation Function paste()

Paste() : This functions basically concatenates two or more strings. It can also be used to concatenate two or more vectors of strings. Strings of the vectors of shorter lengths are concatenated recursively with those of the longest one. The syntax of the function is:

paste(. . . ,sep=" ",collapse=NULL)

Where, . . . implies one or more R objects, to be converted to character vectors

sep is character string to separate the terms.

collapse an optional character string to separate the results.

Paste converts its arguments via **as.character ()** to character strings by **sep**. If the arguments are vectors, they are concatenated term by term to give a character vector result.Thus,the **paste() function** takes multiple elements from the multiple vectors and concatenates them into a single element.

paste0(. . . ,collapse) is equivalent to paste(. . . ,sep=" ",collapse). Thus,the **paste0() function** has space as its default separator.

If a value is mentioned for the **collapse** option, the values in the result are then separated by the value in the concatenated string.

The Paste() function in R...Contd.

Let us check the following R statement with **paste() function** without using the optional parameters:

>**paste('Ramesh','is','a','farmer')**

This statement shows the following output:

[1] "Ramesh is a farmer"

Using paste() with a separator argument

The **sep parameter** in the **paste() function** makes use of the value or the symbols to separate the elements.This is as illustrated below:

>**paste('Ramesh','is','a','farmer',sep='-')**

The above statement returns the following output:

[1] "Ramesh-is-a-farmer"

The paste() function with collapse argument

When the paste() function is applied to a vector, the **sep parameter** does not work.Here comes the **collapse parameter**, which is highly useful while dealing with the vectors. It is used to specify the symbol or values which separate the elements in the vector when the elements are concated into a single element. The following example illustrates this:

The use of paste() function—Contd.

When we write:

>**paste(c('Ramesh','is','a','farmer'),sep='-')**

R shows the following output:

[1] "Ramesh" "is" "a" "farmer"

We observe that the **'sep'** parameter has no effect on the concatenated string;but when we write:

>**paste(c('Ramesh','is','a','farmer'),collapse='-')**

The result returned is:

[1] "Ramesh-is-a-farmer"

which is as desired.

The paste() function with both separator and collapse arguments

We will now see how **sep** and **collapse** arguments will work together.The **sep** value will deal with the values which are to be placed in between the set of elements and the value with the **collapse argument** will make use of specific value to concatenate the elements into single string.The statement:

>**paste(c('Ramesh','is','a','farmer'),1:4,sep='-',collapse='and')**

returns the following output:

The use of paste() function—Contd.

[1] "Ramesh-1andis-2anda-3andfarmer-4"

Use of the paste0() function in R

We know that the **Paste0() function** acts just like paste function but with a default separator space.The following example illustrates its use:

>**paste0('File',1:10)**

[1] "File1" "File2" "File3" "File4" "File5" "File6" "File7" "File8"
[9] "File9" "File10"

Now let us note how **paste0() function** works with the **collapse parameter** as in the example below:

>**paste0('File',1:10,collapse=',')**

The output returned is:

[1] "File1,File2,File3,File4,File5,File6,File7,File8,File9,File10"

Thus, we note that the collapse argument in the paste0() function is the character, symbol, or a value used to separate the elements.

Thus, it is observed that the **paste0() function** returns a string with a default separator and a specified collapse argument as well.

More illustrative uses of paste()

```
>v=c("computer science","computer applications","biological science","physical science","philosophy")
>paste(v,v1)
[1] "computer science RAM" "computer applications SHYAM"
[3] "biological science RAM" "physical science SHYAM"
[5] "phylosophy RAM"
>v2=c("Book","TOOK")
>paste(v,v1,v2)
[1] "computer science RAM Book" "computer applications SHYAM TOOK"
[3] "biological science RAM Book" "physical science SHYAM TOOK"
[5] "phylosophy RAM Book"
>paste("The quick brown"," fox", " jumped over the", " lazy dog")
[1] "The quick brown fox jumped over the lazy dog"
```

The Substr() or substring() and the strsplit()

Substr() or substring(). This function is used to extract character or strings from a given string or a vector of strings for which the starting and the ending position are required to be mentioned. The basic syntax is:

Substr(x, starting-position-number,ending-position number)

Where x is a string or a vector.In case of a vector, the function returns an array of substrings separated from each of the string elements.

The following R statements illustrate the use of the function:

```
substring("Mathematics",3,5)
[1] "the"
>substr(v,2,5) [1] "ompu" "ompu" "iolo" "hysi" "hylo" Here, the vector v is as defined above.
```

strsplit() . This function is used to split a given string or a vector containing strings into a list of substrings where the given 'split-string 'is found. The syntax of the function is:

strsplit(x,"split-string"/variable containing some string)

where x is either a single string or a vector of of strings. The following figure illustrates the application of the function.

```
>isbn="978-93-5023-920-9"
>strsplit(isbn,"-")
[[1]]
[1] "978" "93" "5023" "920" "9"
>strsplit(v,"er")
[[1]]
```

Illustrative Example 6.5 on R User-defined functions

Example-6.5. It is required to develop a function in order to check whether a given string is a **palindrome** or not.

Task Analysis: A given string is said to be a **palindrome** if the reversed form of the string equals the given string. The reversed form can be obtained by using the **substr() function** from the last to the first character as shown in the Code below:

```
# Testing for a Palindrome
str=readline(prompt="Enter any string for testing: ")
l=nchar(str)
rstr=""
for(i in seq(l,1,-1)){
s=substr(str,i,l);l=l-1
rstr=paste(rstr,s,sep="",collapse=NULL)}
if(str==rstr){
print("It is a palindrome")
} else {
print("It is not a palindrome")}
```

Concept of LIST in R

A list is a data structure in which the elements may be of same or different data type. It can also be considered as a special type of vector that can data of same or different types.It is created by using the **list ( )** function as shown below:

```
>l=list(2,3,4,5)
>l
[[1]]
[1] 2
[[2]]
[1] 3
[[3]]
[1] 4
[[4]]
[1] 5
>ls=list("Ram","shyam",40,35,TRUE)
>ls
[[1]]
[1] "Ram"
[[2]]
[1] "shyam"
 [[3]]
[1] 40
[[4]]
[1] 35
[[5]]
```

We can also create a list by assigning names to each of the elements as shown below:

```
>lt=list(Vector=c(2,4,8),Name="Arka Roy", Age=25,List=list(6,'r',T))
```

Concept of LIST in R Contd.

```
>lt
$‘Vector‘
[1] 2 4 8 $Name
[1] "Arka Roy"
$Age
[1] 25
$List
$List[[1]]
[1] 6
$List[[2]]
[1] "r"
$List[[3]]
[1] TRUE
```

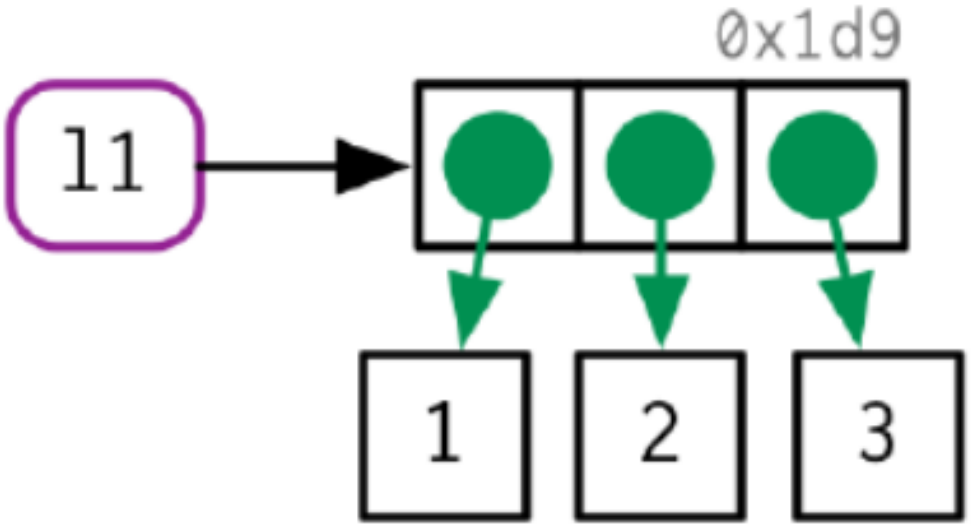
The names can also be assigned in the following ways:

```
names(lt)=c("Vector","Name","Age","An Inner List")
>print(lt)
$‘Vector‘
[1] 2 4 8
 $Name
[1] "Arka Roy"
$Age
[1] 25
$‘An Inner List‘
$‘An Inner List‘[[1]]
[1] 6
$‘An Inner List‘[[2]]
[1] "r"
$‘An Inner List‘[[3]]
[1] TRUE
```

Concept of LIST in R Contd.

Lists are called super data types because they allow different objects, such as matrices, vectors, data frames, and other lists, to be included under one name (the name of the list) in an ordered way. None of these objects needs to be related in any way.So, we can store practically anything in a list.

The internal representation of a list is actually quite different from that of a vector. It is actually a vector of references as depicted below for a list l1 defined as **l1=list(1,2,3)**:



Concept of LIST in R Contd.

The concept is particularly important when we modify a list as shown below: l2=l1 This assignment can be depicted pictorially as shown below:

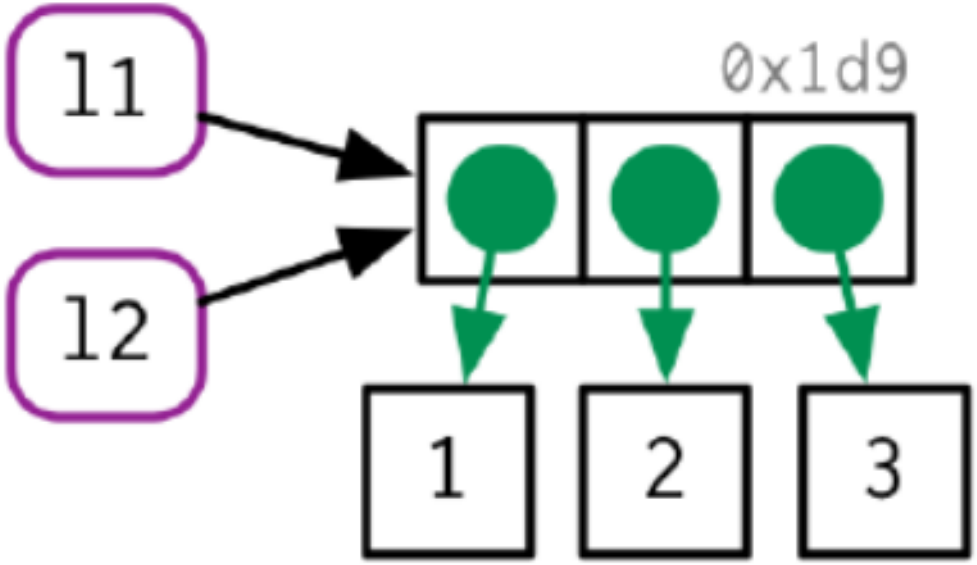


Figure: The internal structure of a list referenced by two different names

Concept of LIST in R Contd.

So,when we write: **l2[[3]] = 4**, the referencing to the values is changed as shown below:

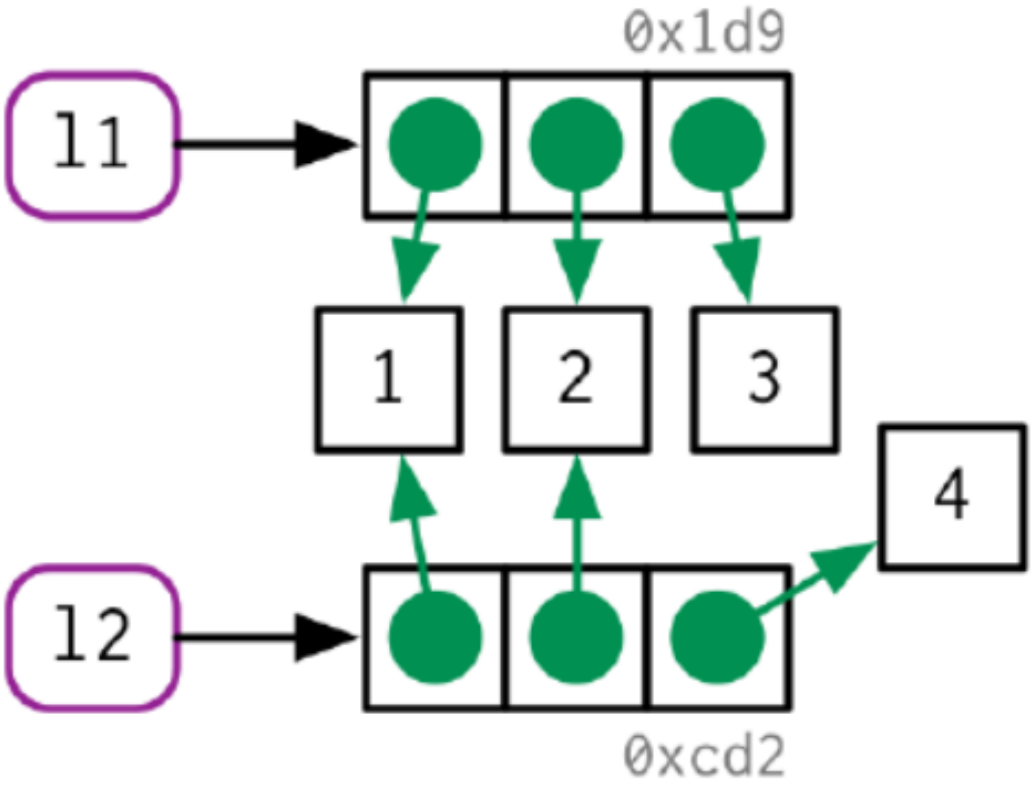


Figure: The internal structure of a list referenced by two different names with common and uncommon values

Creation of an Empty List

An empty list can be created though any one of the following three methods:

1. Using the **list()** function
2. Using the **vector ()** function
3. Storing the **NULL** value in an existing list

Let us have some illustrative examples underneath.

Example-1.A list of length zero.

>**blank**$_list = list()$

>**blank_list**

list()

>**length(blank_list)**

[1] 0

Example-2. A list of any desired length.

>**blank_list2=vector(mode='list', length=5)**

>**blank_list2**

[[1]]
NULL
[[2]]
NULL

Concept of LIST in R Contd.

Like vectors, lists use copy-on-modify behaviour; the original list is left unchanged, and R creates a modified copy. This, however, is a shallow copy: the list object and its bindings are copied, but the values pointed to by the bindings are not. The opposite of a shallow copy is a deep copy where the contents of every reference are copied. Prior to R 3.1.0, copies were always deep copies.

R uses references with character vectors also.

Let us take a vector x defined as:

x = c("a", "a", "abc", "d")

Then, we may represent it pictorially as shown below:

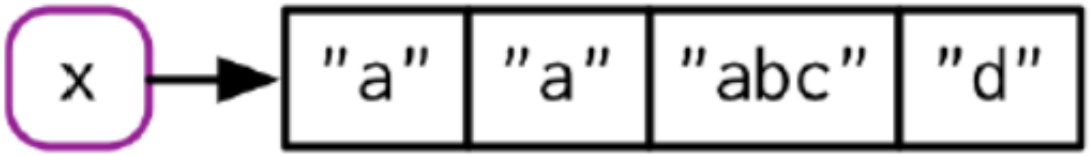


Figure: The internal structure of a character vector

Pictorial Presentation of a character vector

R actually uses a global string pool where each element of a character vector is a pointer to a unique string in the pool as shown below:

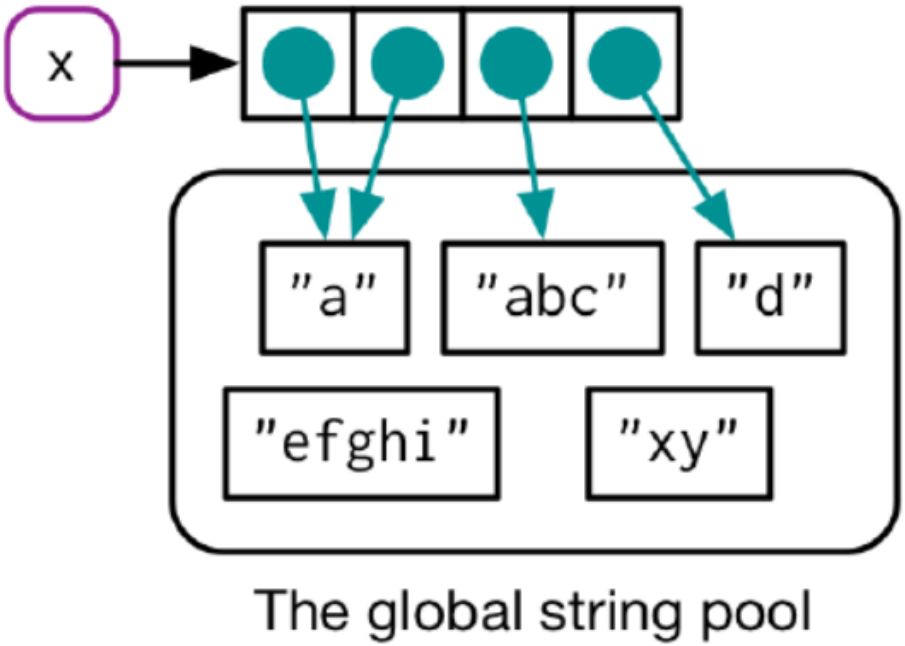


Figure: The internal structure of a character vector

We can request that **ref()** show these references by setting the character argument to **TRUE**:

ref(x, character = TRUE)

Accessing the components of a list

The elements of a list can be accessed by using subscripts as illustrated below :

```
>lt[[1]]
[1] 2 4 8
>lt[[2]]
[1] "Arka Roy"
>lt[[3]]
[1] 25
>lt[[4]]
[[1]]
[1] 6
[[2]]
[1] "r"
[[3]]
[1] TRUE
>lt[[5]]
Error in lt[[5]] : subscript out of bounds
```

The number of subscripts must be less than or equal to the number of elements in a list.

The subscripts may also be written within single pair of square brackets to get the same output as illustrated below:

```
>lt[1]
$`Vector`
[1] 2 4 8
>lt[2]
$`Name`
[1] "Arka Roy"
>lt[3]
$`Age`
[1] 25
>lt[4]
$`An Inner List
` $`An Inner List`[[1]]
[1] 6
```

Accessing Contd.

```
$`An Inner List`[[2]]
[1] "r"
$`An Inner List`[[3]]
[1] TRUE
```

We observe that the components of a list can be broken down into smaller components but the same thing cannot be done in case of a vector. So, **the normal vectors that have been defined earlier can be called atomic vectors**, whereas, the **lists can be called recursive vectors**.

Manipulation of the elements of a list

We can enhance a list by adding values to the desired location. Here, the location is too mentioned by the name of the list followed by the subscript of the desired location within square brackets ([ ]). If the subscript mentions a location which is far outside the current list, the skipped locations will contain a special value called NULL which means value is not available in the location. If we mention some location that already contains some value, that value will be replaced with the new value. These are illustrated in the machine sessions below:

```
>lt[1]=5 # This replaces the current value at the first location of the list.
>lt  # Let us now have a look at the elements of the list
$`Vector`
[1] 5
```

Manipulation of the elements of a list–Contd.

```
$Name
[1] "Arka Roy"
$Age
[1] 25
$`An Inner List`
$`An Inner List`[[1]]
[1] 6
$`An Inner List`[[2]]
[1] "r"
$`An Inner List`[[3]]
[1] TRUE
>lt[5]="Latest Value"  # Enhancing the elements in the list by putting a new one.
>lt[6]='Next Value'  # Enhancing the elements in the list by putting another new value.
>lt  # Listing the current elements of the list.
 [1] 6
$`An Inner List`[[2]]
[1] "r"
$`An Inner List`[[3]]
[1] TRUE
[[5]]
[1] "Latest Value"
[[6]]
[1] "Next Value"
>lt[8]="A New element" # Inserting a value at the 8th location when the list has elements upto the 6th location.
>lt[7] # As the there is no value at the 7th location, R shows NULL.
[[1]]
NULL
$`Vector`
[1] 5
$Name
[1] "Arka Roy"
$Age
[1] 25
$`An Inner List`
```

Manipulation of the elements of a list–Contd.

```
$`An Inner List`[[1]]
[1] 6
$`An Inner List`[[2]]
[1] "r"
$`An Inner List`[[3]]
[1] TRUE
[[5]]
[1] "Latest Value"
[[6]] [1] "Next Value"
>lt[2]='Pranab Roy' # Changing an existing value
>lt # listing again
$`Vector`
[1] 5
$Name
[1] "Pranab Roy"
$Age
[1] 25
$`An Inner List`
$`An Inner List`[[1]]
```

We can also delete an element from a list by storing NULL in the location as shown below:

```
>lt[1]=NULL
>lt[1] # This shows the value which was previously at location 2 because the value at location 1 is not available now and the value of the second location comes to the first location and similar phenomenon occurs for the remaining locations reducing the size of the list by one.
$`Name`
[1] "Pranab Roy"
>lt[5]=NULL
>lt
$`Name`
[1] "Pranab Roy"
$Age
[1] 25
$`An Inner List`
$`An Inner List`[[1]]
```

Manipulation of the elements of a list–Contd.

```
[1] 6
$`An Inner List`[[2]]
[1] "r"
$`An Inner List`[[3]]
[1] TRUE
[[4]]
[1] "Latest Value"
[[5]]
NULL
[[6]]
[1] "A New element"
```

Another manipulation of lists can be done by merging multiple lists. This can be done by using the **list ()** function. This is illustrated below:

```
l=list(12,45,67,c('A','B','C'))
>merged_list=list(lt,l)
>merged_list
[[1]]
[[1]]$`Age`
[1] 25
[[1]]$`An Inner List`
[[1]]$`An Inner List`[[1]]
[1] 6
[[1]]$`An Inner List`[[2]]
[1] "r"
[[1]]$`An Inner List`[[3]]
[1] TRUE
[[1]][[3]]
[1] "Latest Value"
[[1]][[4]]
NULL
[[1]][[5]]
[1] "A New element"
```

Conversion of lists to vectors

A list can be converted into a vector by using the **unlist()** function when all the values of the list will be upgraded to the same data type as mentioned earlier in case of vectors. This is illustrated in the following listing of commands and the values of the vectors.

```
>v1=unlist(lt)
>v2=unlist(v2)
>print(v1)
 Age An Inner List1 An Inner List2 An Inner List3
 "25" "6" "r" "TRUE" "Latest Value" "A New element"
>v2
[1] "A" "B" "C"
```

Factors in R

In R, a **factor** is a data structure **used for predefined, finite number of values**. The data values are called **categorical data**. Categorical data **are also known as nominal or qualitative data**. A variable representing a categorical data is called a **categorical or nominal variable**. Examples of categorical or nominal variables are colors (**Violet, Indigo, Blue, Red**), Gender (**Male, Female, Other**), Nationality (**Indian, British, French**) , Response(**Yes, No**), Marital-Status(**Single, Married, Divorced, Widowed**) etc. The label or description of a categorical variable is called its level. The levels cannot be ordered. It is because of the insensibility of assigning numerical values to the different categories and hence ordering on the basis of that. However, the levels may be numbered arbitrarily for ease of handling. In R, **factors may be defined as special vectors that hold categorical data**. **The function factor () transforms a vector into a factor**. The factor stores the nominal values as a vector of integers in the range [1... n] (where n is the number of unique values in the nominal variable), and an internal vector of character strings (the original values) are mapped to these integers. Let us now have an illustrative session for factors in R.

An illustrative session for factors in R

```
M_Status=c(rep("single",20),rep("Married",30),rep("Divorced",10))
>Marital_Status=factor(M_Status)
>Marital_Status
[1] single single single single single single single single single
[10] single single single single single single single single single
[19] single single Married Married Married Married Married Married Married
[28] Married Married Married Married Married Married Married Married Married
[37] Married Married Married Married Married Married Married Married Married
[46] Married Married Married Married Married Divorced Divorced Divorced Divorced
[55] Divorced Divorced Divorced Divorced Divorced Divorced
Levels: Divorced Married single
```

R now treats Marital_Status as ordinal variable. R associates 1s to 10 Divorced values, 2s to 30 Married values and 3s to 20 Single values. This is done in alphabetical order of the categorical values namely, Divorced, Married and Single. If we want to see the summarized value of the factor, we use the **summary() function** as shown below:

```
>summary(Marital_Status)
   Divorced     Married      single
     10          30          20
```

If we want to see the different labels of the categorical values, we use the **levels() function** as illustrated below. To see the internal structure of an R object, we use the **str() function**, Let us see their effects below:

```
>levels(Marital_Status)
[1] "Divorced" "Married" "single"
>str(Marital_Status)
 Factor w/ 3 levels "Divorced", "Married",..: 3 3 3 3 3 3 3 3 3 3
```

Data Frames in R

A data frame in R is a two dimensional tabular representation of data in rows and columns where columns are of same size with same or different data types called **mode**. It is a natural way of representing data from relational database tables, Excel sheets, vectors or factors of equal length etc. data sets. The data sets may either be qualitative or quantitative i.e. categorical or numerical. It is like a list with components as columns of a table. Columns in a data frame are usually named which are often referenced as variables. The rows of a data frame may also be named, if the user likes. The **data. frame ( ) function** is used to create a data frame.

The following R statement shows the creation of a data frame.

data.frame(Name=c("Ram","Ramesh","Mamata"),Marital_Status=c("Married","Divorced","Single"),Age=c(35,37,62))

	Name	Marital_Status	Age
1	Ram	Married	35
2	Ramesh	Divorced	37
3	Mamata	Single	62

Data Frames in R–Salient Points to be borne in mind

A **data frame** is a table of data or a two-dimensional array-like structure in which each column contains values of one variable represented by a column name and each row contains one set of values for each column variable. The **common characteristics** of a data frame may be pointed out briefly as under:

- The column names should be **non-empty**.
- The row names should be **unique**.
- The data stored in a data frame can be either of **numeric, factor or character** type.
- Each column being an atomic vector with a **common width** should contain the same number of data items of **same data type**
- It can be **expanded** by adding columns and rows
- It can be initially defined as an **empty one** also so that data can be filled in later conveniently

Empty DataFrames

An **empty dataframe** is an initialized dataframe without any rows.
There are two basic methods of creating empty dataframes in R. They are:

1. By creating a matrix with column names
2. By initializing columns with empty vectors

Let us illustrate method 1 first.
Here, we create an empty data frame by using the following steps:

- We first define a matrix with 0 rows and as many columns as we would like.
- Then we use the **data.frame()** function to convert it to a data frame and the **colnames()** function to give it column names.
- Then we use the **str()** function to analyze the structure of the resulting data frame.

In R,we proceed as under:

```
>df=data.frame(matrix(ncol=4,nrow=0))
>colnames(df)=c('Column_Name1','Column_Name2',
'Column_Name3','Column_Name4')
```

Empty DataFrames–Contd.

```
>df
[1] Column_Name1    Column_Name2    Column_Name3    Column_Name4
<0 rows>(or 0-length row.names)
>str(df)
'data.frame': 0 obs. of 4 variables:
$ Column_Name1: logi
$ Column_Name2: logi
$ Column_Name3: logi
$ Column_Name4: logi
```

The second method is basically the task of making the specification of the data types(class types) for each column and naming them, but having no rows contained in the dataframe.To be more specific, we follow the following steps:

- We define a data frame as a set of empty vectors with specific class types.
- We next specify **stringsAsFactors=False** so that any character vectors are treated as strings, not factors.

Empty DataFrames–Contd.

To illustrate the concept in R,let us first create an empty dataframe with three columns comprising **Date**,**File** and **User**. This can be stated as under:

df = data.frame(Date=as.Date(character()),File=character(), User=character(),stringsAsFactors=FALSE)
>**str(df)**
'data.frame': 0 obs. of 3 variables:
$ Date: 'Date' num(0)
$ File: chr
$ User: chr
In general, it is the task of initializing the desired columns of the dataframe with empty vectors.
The following example illustrates the creation of a dataframe with five empty vectors:
df = data.frame(Doubles=double(),Ints=integer(),Factors=factor(), Logicals=logical(), Characters=character(),stringsAsFactors=FALSE)

Empty DataFrames–Contd.

>**str(df)**
'data.frame': 0 obs. of 5 variables:
$ Doubles : num
$ Ints : int
$ Factors : Factor w/ 0 levels:
$ Logicals : logical
$ Characters: chr

If there is already an existent data frame, say **df**, that has the columns we want, then we can just create an empty data frame by removing all the rows by using the statement as stated below:

empty_df = df[FALSE,]

It may be noticed that **df** still contains the data, but **empty_df** doesn't.

Number of rows and columns in a data frame

We can use the **nrow()** function to know the number of names in a data frame. Similarly,the **ncol()** function can be used to know the number of columns.

Illustrative Data frames

Let us consider creating the following dataframe:

social_status=data.frame(Persons=c("Ram","shyam","Jadu"),
Monetary_status=c("Poor","Middle class","Rich"),
Educational_status=c("Highly Educated","Moderately Educated",
"Lowly_educated"),Age=c(30,40,50))

>**social_status**

	Persons	Monetary_status	Educational_status	Age
1	Ram	Poor	Highly Educated	30
2	shyam	Middle class	Moderately Educated	40
3	Jadu	Rich	Lowly_educated	50

To see the dimension of the data frame, we use the **dim()** function as illustrated below:

>**dim(social_status)**

R shows the following for this statement:

[1] 3 4

Entries from a data.frame can be pointed out with subscripts written within square brackets and separated by comma implying row number followed by column number with the variable holding the data.frame. This is illustrated in the figure below:

>**social_status[1,3]**
[1] Highly Educated
Levels: Highly Educated Lowly_educated Moderately Educated
>**social_status[3,2]**

Insertion of Rows and Columns in a R Data Frame

[1] Rich
Levels: Middle class Poor Rich
New components can be inserted into a data.frame, if required, as shown below:
social_status$Bank_Balance=c(10000,80000,500000)
>social_status

	Persons	Monetary_status	Educational_status	Age	Bank_Balance
1	Ram	Poor	Highly Educated	30	1e+04
2	shyam	Middle class	Moderately Educated	40	8e+04
3	Jadu	Rich	Lowly_educated	50	5e+05

We may also add further rows in an existing data.frame. This can be done by defining additional row values and then using a function named **rbind( )** as illustrated below.

>ss=data.frame(Persons=c("Madhu","Jidu","Sidhu"),Monetary_status=c("Poor","poor","Rich"), Educational_status=c("Highly Educated","Lowly Educated","Moderately Educated"),Age=c(35,45,55),Bank_Balance=c(25000,35000,90000))
>ss=rbind(social_status,ss)
>ss

	Persons	Monetary_status	Educational_status	Age	Bank_Balance
1	Ram	Poor	Highly Educated	30	10000
2	shyam	Middle class	Moderately Educated	40	80000
3	Jadu	Rich	Lowly_educated	50	500000
4	madhu	Poor	Highly Educated	35	25000
5	Jidu	poor	Lowly Educated	45	35000
6	Sidhu	Rich	Moderately Educated	55	90000

Insertion of Columns in a R Data Frame Contd.

New columns can also be inserted into a data frame by using a column binding function named **cbind()**. We illustrate the concept below:

```
Marital_Status=c("MARRIED","MARRIED","MARRIED","MARRIED","Single","Divorced")
>cbind(ss,Marital_Status)
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	MARRIED
2	shyam	Middle class	Moderately Educated	40	80000	MARRIED
3	Jadu	Rich	Lowly_educated	50	500000	MARRIED
4	madhu	Poor	Highly Educated	35	25000	MARRIED
5	Jidu	poor	Lowly_Educated	45	35000	Single
6	Sidhu	Rich	Moderately Educated	55	90000	Divorced

```
>final_ss=cbind(ss,Marital_Status)
>final_ss
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	MARRIED
2	shyam	Middle class	Moderately Educated	40	80000	MARRIED
3	Jadu	Rich	Lowly_educated	50	500000	MARRIED
4	madhu	Poor	Highly Educated	35	25000	MARRIED
5	Jidu	poor	Lowly_Educated	45	35000	Single
6	Sidhu	Rich	Moderately Educated	55	90000	Divorced

```
>Marital_Status=c(TRUE,TRUE,FALSE,FALSE,TRUE,TRUE)
>cbind(ss,Marital_Status)
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
3	Jadu	Rich	Lowly_educated	50	500000	FALSE
4	madhu	Poor	Highly Educated	35	25000	FALSE
5	Jidu	poor	Lowly_Educated	45	35000	TRUE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE

Addition of Columns in a R Data Frame Contd.

There is still a third way of adding new columns into a data frame as illustrated below:

```
ss[["Marital_Status"]]=Marital_Status
>ss
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
3	Jadu	Rich	Lowly_educated	50	500000	FALSE
4	madhu	Poor	Highly Educated	35	25000	FALSE
5	Jidu	poor	Lowly_Educated	45	35000	TRUE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE

However, the **cbind( )** function can be used to add more than one column at a time into an existing data frame as illustrated below:

```
x=c(1,2,3,4,5,6)
>y=c(7,8,9,10,11,12)
>cbind(ss,x,y)
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status	x	y
1	Ram	Poor	Highly Educated	30	10000	TRUE	1	7
2	shyam	Middle class	Moderately Educated	40	80000	TRUE	2	8
3	Jadu	Rich	Lowly_educated	50	500000	FALSE	3	9
4	madhu	Poor	Highly Educated	35	25000	FALSE	4	10
5	Jidu	poor	Lowly_Educated	45	35000	TRUE	5	11
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE	6	12

Now, if we want to **change the column name Marital_Status** to **Married**, we need to write the statement in the following way:

```
>colnames(ss)[which(names(ss)=="Marital_Status")]="Married"
```

Deletion of columns from a Data Frame

To remove an entire column from a data.frame in R,we can follow any one of the approaches illustrated below:

1. Storing **NULL** in the desired column of the data frame **ss** defined initially with 3 rows:

>**ss$Age=NULL**

Let us now check the deletion: >**ss**

	Persons	Monetary_status	Educational_status	Bank_Balance
1	Madhu	Poor	Highly Educated	25000
2	Jidu	poor	Lowly_Educated	35000
3	Sidhu	Rich	Moderately Educated	90000

Let us redefine the data frame as under,

>**ss=data.frame(Persons=c("Madhu","Jidu","Sidhu"), Monetary_status=c("Poor","poor","Rich"),Educational_status= c("Highly Educated","Lowly Educated","Moderately Educated"),Age= c(35,45,55),Bank_Balance=c(25000,35000,90000))**

2. The following example shows the storing of NULL value using the sub-setting option:

>**ss[[4]]=NULL**

>**ss**

Deletion of columns from a Data Frame–Contd.

	Persons	Monetary_status	Educational_status	Bank_Balance
1	Madhu	Poor	Highly Educated	25000
2	Jidu	poor	Lowly_Educated	35000
3	Sidhu	Rich	Moderately Educated	90000

3. The subsetting can also be done in the following way:

>**ss[2]=NULL**

To check the outcome, we write:

>**ss**

This shows the following table:

	Persons	Educational_status	Age	Bank_Balance
1	Madhu	Highly Educated	35	25000
2	Jidu	Lowly_Educated	45	35000
3	Sidhu	Moderately Educated	55	90000

4. Use of negative column number in the subsetting also removes the column as shown below:

>**ss= ss[,-3]**

>**ss**

This removes the **Age** column.

The following illustrated format also removes the 3rd column:

>**ss=ss[-3]**

Deletion from a Data Frame–Contd.

5. For deletion of multiple columns at a time, we need to use a list of **NULL** values as stated below:

>**ss[2:4]=list(NULL)**

>**ss**

This shows the following table:

	Persons	Bank_Balance
1	Madhu	25000
2	Jidu	35000
3	Sidhu	90000

6. Deletion by using the column names is to be done as shown below:

>**ss=subset(ss,select=-Educational_status)**

For deletion of rows,we use the index number of the row as shown below:

>**ss=ss[-1,]**

>**ss**

	Persons	Monetary_Status	Bank_Balance
2	Jidu	poor	35000
3	Sidhu	Rich	90000

For the removal of multiple rows at a time, we make use of the subset function as shown below:

>**subset(ss,Monetary_status!="Poor"&Age>50)**

Insertion of columns at any desired position of a dataframe

We have observed that **cbind() function** enables us to add a column always after the existing columns of a dataframe. Now, if we want to insert a column at any desired column position in a dataframe, we can do it by using an in-built R function add_column() having the following syntax:

add_column(Name_of_existing_dataframe,New_column_definition,.before|.after

Here, option-1|option-2 implies that any one of the options is to be used and the braces are not part of the actual use in a statement.

Here, we shall need to install the **'tibble'** package and its library shall have to be loaded before the use of the function. An illustrative example has been shown below:

>**library(tibble)**

>**df=data.frame(A = 10:14, B = 21:25, C=33:37)**

This creates the following dataframe:

	A	B	C
1	10	21	33
2	11	22	34
3	12	23	35
4	13	24	36
5	14	25	37

Insertion of columns and rows at any desired position of a dataframe

Let us now insert a new column after the column number 2. We issue the following command for this purpose:

```
>df=add_column(df, D = 43:47, .after = 2))
>df
```

	A	B	D	C
1	10	21	43	33
2	11	22	44	34
3	12	23	45	35
4	13	24	46	36
5	14	25	47	37

Now, to insert a row at any desired row position of a dataframe,we develop a function as shown below:

```
insertRow = function(existingDF, newrow, row_number) {
existingDF[seq(r+1,nrow(existingDF)+1),] =
existingDF[seq(r,nrow(existingDF)),]
 existingDF[r,] = newrow
existingDF
 }
```

Insertion of rows at any desired position of a dataframe

Let us use the function as illustrated below with the dataframe **df** defined above.

>**r=2**

>**rw=seq(4)**

>**insertRow(df,rw,r)**

This generates the following dataframe with the newly inserted row:

	A	B	D	C
1	10	21	43	33
2	1	2	3	4
3	11	22	44	34
4	12	23	45	35
5	13	24	46	36
6	14	25	47	37

We can also assign the resulting dataframe to the existing one to make the changes permanent in the dataframe.

>**df=insertRow(df,rw,r)**

Retrieval of data from a dataframe depending on requirement

Let us consider the dataframe **airquality** that comes with the R system and readily available for use.

Let us first check its structure as shown below:

>**str(airquality)**

'data.frame': 153 obs. of 6 variables:

$ Ozone : int 41 36 12 18 NA 28 23 19 8 NA ...

$ Solar.R: int 190 118 149 313 NA NA 299 99 19 194 ...

$ Wind : num 7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...

$ Temp : int 67 72 74 62 56 66 65 59 61 69 ...

$ Month : int 5 5 5 5 5 5 5 5 5 5 ...

$ Day : int 1 2 3 4 5 6 7 8 9 10 ...

Now,to retrieve the data of the first five rows from the dataframe,we write:

>**airquality[1:5,]**

This gives us the following result:

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5	NA	NA	14.3	56	5	5

Now,if we want to see the data of the columns 2 to 4 for the first 3 rows, we write the following:

>**airquality[1:3,2:4]**

This gives us the following output:

	Solar.R	Wind	Temp
1	190	7.4	67
2	118	8.0	72
3	149	12.6	74

Customized Retrieval of data from a dataframe Contd.

Let us now retrieve the data of row numbers:1,5 and 9. For this purpose, we write the following statement:
>**airquality[c(1,5,9),]**
It may be noted that we have used here a vector comprising the desired row numbers.
This gives us the following results:

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
5	NA	NA	14.3	56	5	5
9	8	19	20.1	61	5	9

We may like to change the NA values in the dataframe with some specific value for easy recognition,say, 999.We can do this with the following command:
>**airquality[is.na(airquality)]=999**
Let us check the effect on the NA values got in the output of the preceding query:
>**airquality[c(1,5,9),]**
The following result is generated:

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
5	999	999	14.3	56	5	5
9	8	19	20.1	61	5	9

Let us now retrieve the data of the columns 3 and 5 for the rows 5,7 and 13.The statement written below serves the purpose.
>**airquality[c(5,7,13),c(3,5)]**
It may be observed here that both row numbers and column numbers are not sequential,rather randomly chosen;hence, we need to express them as vectors as shown below:

	Wind	Month
5	14.3	5
7	8.6	5
13	9.2	5

Splitting a Data Frame in R

We can split a data from on some variable of the data frame so that we can obtain sub dataframes for each distinct value of the variable. The **split()** function is used for this purpose with the following syntax:

split(Name-of-the data frame, Name-of-data-frame$ column or variable-name-of the data frame)

This is illustrated below:

```
>split(social_status,social_status$monetary_status)
$Middleclass
 persons monetary_status educational_status        Age
2 shyam Middleclass        Moderately Educated      40
$Poor
 persons monetary_status educational_status Age
1 Ram        Poor        Highly Educated      30
$Rich
persons monetary_status educational_status     Age
3 Jadu        Rich        Lowly_educated        50
>split(social_status,social_status$educational_status)
$'Highly Educated'
 persons monetary_status educational_status     Age
1 Ram        Poor        Highly Educated        30
$Lowly_educated
 persons monetary_status educational_status     Age
3 Jadu        Rich        Lowly_educated        50
$'Moderately Educated'
persons monetary_status educational_status        Age
2 shyam Middle class        Moderately Educated      40
```

Performing queries in a data frame

We may like to select some of the rows from a data frame that satisfy a given condition. The condition, however, should be based on some of the column value. One or more conditions may be stated; but while using the logical operators, the user should keep it mind that **only elementwise logical operators** can be used. The following codes illustrate a few queries which are self explanatory.

```
>subset(ss,ss$Bank_Balance>=50000)
 Persons Monetary_status Educational_status Age Bank_Balance
3 Sidhu Rich Moderately Educated 55 90000
>rich=ss[ss$Bank_Balance>=50000,]
>rich
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
2	shyam	Middle class	Moderately Educated	40	8e+04	TRUE
3	Jadu	Rich	Lowly_educated	50	5e+05	FALSE
6	Sidhu	Rich	Moderately Educated	55	9e+04	TRUE

```
>ss[ss$Bank_Balance<50000,]
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE
4	madhu	Poor	Highly Educated	35	25000	FALSE
5	Jidu	poor	Lowly Educated	45	35000	TRUE

```
>subset(ss,Educational_status=="Highly Educated")
Persons Monetary_status Educational_status Age Bank_Balance
1 Madhu Poor Highly Educated 35 25000
```

Performing queries in a data frame–Contd.

```
y=ss[ss$Bank_Balance<50000 & ss$Marital_Status,]
>y
```

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE

Obviously, the format of the conditional expression is:

>Data-frame-name[data-frame-name$column-name relational-operator value-to-be-tested{elementwise-logical-operator data-frame-name$ column-name relational-operator value-to-be-tested. . . . . .},]

The portion within the braces() is optional. If it is used, it is to be used without the curly brackets. The **ellipsis(. . . ..)** implies that the preceding option can be repeated any number of times.

Sorting Data Frames

We can sort a data frame by a selected column, either in ascending sequence or in descending sequence of the stated column values. The syntax of the command is:

Name of the data-frame[rev(order(name-of- the -data-frame[, "Column-name"])),]

Here, **Column-name** refers to the column on the basis of the values of which all the rows of the data-frame will be sorted.Continued.....

Sorting Data Frames–Contd.

The **rev** option ,if specified, the sorting will be in descending or decreasing order. The sorted data frame can first be assigned to another data frame variable , so that it can be seen later and repeatedly, if required. The following codes illustrate the concept.

ss[order(ss[,"Bank_Balance"]),]

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE
4	madhu	Poor	Highly Educated	35	25000	FALSE
5	Jidu	poor	Lowly Educated	45	35000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE
3	Jadu	Rich	Lowly_educated	50	500000	FALSE

>**ss[rev(order(ss[,"Bank_Balance"])),]**

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
3	Jadu	Rich	Lowly_educated	50	500000	FALSE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
5	Jidu	poor	Lowly Educated	45	35000	TRUE
4	madhu	Poor	Highly Educated	35	25000	FALSE
1	Ram	Poor	Highly Educated	30	10000	TRUE

Merging data Frames: Merging of data frames implies combining two data frames to generate a new data frame. This is equivalent to **Join** operation of relational algebra.

Merging of data Frames

Similar to different forms of Join, we can merge two data frames for results of **Equijoins**,**Right Outer Join**, **Left Outer Join** and **Cross Join or Cartesian product**. The syntax is:

merge(df1,df2[,by=$\left\{ \begin{array}{l} \text{"common column name"} \\ \text{NULL} \end{array} \right\}$**][,all[.**$\left\{ \begin{array}{l} \text{x} \\ \text{y} \end{array} \right\}$**]=TRUE])**

Here, options within square brackets([]) imply that their use is optional.Options within a pair of braces imply that one of them is required to be used.

To illustrate the use of the function in different ways, we define two simple data frames as under:

```
>df1=data.frame(EMPNO=c(1,5,7,8,9))
>df2=data.frame(EMPNO=c(3,5,7,4,2))
```

Now,we perform equi-join as under:

```
>merge(df1,df2)
```

	EMPNO
1	5
2	7

We perform **Left Outer Join** as shown below:

```
>merge(df1,df2,by="EMPNO",all.x=TRUE)
```

The generated output is:

	EMPNO
1	1
2	5
3	7
4	8
5	9

Merging of data Frames–Contd.

For the **Right Outer Join**, we write the command as under:

>**merge(df1,df2,by="EMPNO",all.y=TRUE)**

This gives the output shown below:

	EMPNO
1	2
2	3
3	4
4	5
5	7

For the **Full outer Join**, we write the following command:

>**merge(df1,df2,by="EMPNO",all=TRUE)**

The following is the generated output:

	EMPNO
1	1
2	2
3	3
4	4
5	5
6	7
7	8
8	9

Merging of data Frames–Contd.

For the **Cross Join**, we write the following command:
>merge(df1,df2,by=NULL)
The generated result of **Cartesian Product**,also called **CrossJoin** of the two data frames is as shown below:

	EMPNO.x	EMPNO.y
1	1	3
2	5	3
3	7	3
4	8	3
5	9	3
6	1	5
7	5	5
8	7	5
9	8	5
10	9	5
11	1	7
12	5	7
13	7	7
14	8	7
15	9	7
16	1	4
17	5	4
18	7	4
19	8	4
20	9	4
21	1	2
22	5	2
23	7	2
24	8	2
25	9	2

Sorting Data Frames–Contd.

We may also like to see the values of a particular column in a sorted order. This can be achieved by using the **sort( ) function** as illustrated below:

sort(ss$Bank_Balance)

[1] 10000 25000 35000 80000 90000 500000

To effect the ordering of the rows in the data frame also, we need to use **order( )** function on the data of the column to obtain the ranks of the values first i.e. the row positions of the values in the increasing order and then we can utilize that **ranking** according to the syntax given below:

First, we find out the ranking as shown below:

>**Ranking = order(name-of-data-frame $ Name-of-column)**

Next , we write:

>**Name-of-data-frame[Ranking,]**

This is illustrated below:

ranking=order(ss$Bank_Balance)

>**ranking**

[1] 1 4 5 2 6 3

>**ss[ranking,]**

Ranking Data Frames–Contd.

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
1	Ram	Poor	Highly Educated	30	10000	TRUE
4	madhu	Poor	Highly Educated	35	25000	FALSE
5	Jidu	poor	Lowly Educated	45	35000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE
3	Jadu	Rich	Lowly_educated	50	500000	FALSE

The ranks printed imply that the first value occurs in the first row, the second value in the order occurs in the row number 4 and so on. The reverse ordering can now be obtained as shown below:

>ss[order(ss$Bank_Balance,decreasing=TRUE),]

	Persons	Monetary_status	Educational_status	Age	Bank_Balance	Marital_Status
3	Jadu	Rich	Lowly_educated	50	500000	FALSE
6	Sidhu	Rich	Moderately Educated	55	90000	TRUE
2	shyam	Middle class	Moderately Educated	40	80000	TRUE
5	Jidu	poor	Lowly Educated	45	35000	TRUE
4	madhu	Poor	Highly Educated	35	25000	FALSE
1	Ram	Poor	Highly Educated	30	10000	TRUE

If we want to sort the dataframe on the basis of multiple columns,one under the other in the order mentioned, we can do it by using the **with option** as illustrated below for **Persons** under **Bank_Balance**.

>ss= ss[with(ss, order(Persons, Bank_Balance)),]

>ss[with(ss, order(Persons, Bank_Balance,decreasing=TRUE)),]

Illustration of head() and tail() for a data frame df

Inspecting the rows of a dataframe

The first six rows of a data frame can be inspected by using the **head()** function.For the last six rows, the **tail()** function is used. The syntax of the functions are:

- **head(name_of_dataframe[,n])**,where n is the number of rows to be displayed from the beginning which is 6 ,by default
- **tail(name_of_dataframe[,n])**,where n is the number of last rows to be displayed which is 6 ,by default

>**head(df)**

	term	count
1	label	103
2	book	55
3	one	40
4	game	38
5	love	34
6	great	29

>**head(df,3)**

	term	count
1	label	103
2	book	55
3	one	40

>**tail(df,2)**

	term	count
1687	remast	1
1688	rerecord	1

The setdiff() and Conclusion on data frame

The **setdiff()** function

This function is used to find out the elements in two vectors or data frames which is in the first vector or data frame(minuend), but not in the second vector or data frame(subtrahend). Its use has been illustrated below:

x=c(1:20)

>y=c(15:40)

setdiff(x,y)

[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14

This function can also be used to show the rows of the first dataframe which do not belong to the second one.

Format:setdiff(df1,df2)

Further important points about dataframe to be borne in mind.

- By default data frames turn strings into factors. So, we should use **stringsAsFactors = FALSE** to suppress this behaviour:
- We can **coerce an object** to a data frame with **as.data.frame()**:
  1. **A vector will create a one-column data frame**.
  2. **A list will create one column for each element**; it's an error if they're not all the same length.
  3. **A matrix** will create a data frame with the same number of columns

Extended Data Structures-Matrices

A **matrix** is a two dimensional version of a numeric vector. The entries in a matrix M are arranged as a rectangular set of data of single type (although useless, character and logical values can also be put in a matrix form) having rows and columns. Matrices are widely used in mathematical calculations. A matrix of order n x m implies that it has n rows and m columns. It is known as the dimension of the matrix. The **matrix( )** function is used to create a matrix. The basic syntax of the **matrix( )** function in R is as stated below:
matrix(data-set,[nrow],[ncol],[byrow], [dimnames]), where,

- **Data-set** is the input vector that becomes the data elements of the matrix,
- **nrow** is used to state the number of rows to be created,
- **ncol** is used to mention the number of columns to be created
- **byrow** specifies whether the data are to be arranged in memory in row-major order or column-major order. Setting byrow to TRUE implies the row-major order. So, the default value is **column-major order**.
- **Dimnames** is used to assign names to the rows and columns.

The first parameter and either of the next two parameters of the **matrix( )** function are mandatory.However,all the parameters of the matrix function may be specified with simply R variables without naming the parameters.

Illustrative examples of the matrix( ) function

Let us have some illustrative examples of the **matrix( )** function.

```
>matrix(10:21,nrow=4)
 [,1] [,2] [,3]
[1,] 10 14 18
[2,] 11 15 19
[3,] 12 16 20
[4,] 13 17 21
>matrix(data=10:21,nrow=4,byrow=TRUE)
 [,1] [,2] [,3]
[1,] 10 11 12
[2,] 13 14 15
[3,] 16 17 18
[4,] 19 20 21
>matrix(data=10:21,ncol=4)
 [,1] [,2] [,3] [,4]
[1,] 10 13 16 19
[2,] 11 14 17 20
[3,] 12 15 18 21
>matrix(data=scan(n=12,quiet=TRUE),nrow=4)
1: 10
2: 11
3: 12
4: 13
5: 14
6: 15
7: 16
8: 17
9: 18
10: 19
11: 20
12: 21
```

As soon as the entry of the inputs becomes over, the following matrix form of values are displayed by R.

Displaying the values in a matrix

```
 [,1] [,2] [,3]
[1,] 10 14 18
[2,] 11 15 19
[3,] 12 16 20
[4,] 13 17 21
```

The function **dim(x)**, where x is the variable to which we assign a matrix, is an integer vector giving the number of rows and columns of the matrix x; i.e. **dim()** function gives the dimension of the matrix. The following depicts its use.

```
>x=matrix(1:12,4)
>x
 [,1] [,2] [,3]
[1,] 1 5 9
[2,] 2 6 10
[3,] 3 7 11
[4,] 4 8 12
>dim(x)
[1] 4 3
>x[1,]
[1] 1 5 9
>x[,2]
[1] 5 6 7 8
>x[3,3]
[1] 11
>i=scan(n=1) # to accept the row number as input
1: 4
>j=scan(n=1,quiet=TRUE) # This to accept the column number as input
1: 2
>x[i,j] # row and column numbers are being expressed with variable values.
[1] 8
>class(x)
[1] "matrix"
>class(x[1,])
[1] "integer"
>class(x[i,j])
[1] "integer"
```

R inputs into a matrix

R script for generalized inputs

```
# matinput.r
cat("How many rows?")
r=scan(n=1,quiet=TRUE)
 cat("How many Columns?")
 c=scan(n=1,quiet=TRUE)
tot=r*c
cat("Enter the matrix element rowwise for",r,"rows and ",c,"Columns
and press enter for each value")
mat=matrix(scan(n=tot),r,byrow=TRUE)
cat("The input matrix is shown below:")
 for( i in 1:r){  for (j in 1:c){
cat(mat[i,j],' ')}
cat(")
}
```

Matrix Manipulations

Let x be a matrix with normal numeric values. Let l be another matrix with logical values. Then, x[l] will return a numeric vector with position numbers in the matrix x, considering columnwise from 1 to n, where l contains the TRUE value. If the dimensions are not matching, then it return NA , where values are not available. The following figure illustrates the idea.

```
>x=matrix(1:4,2)
>x
 [,1] [,2]
[1,] 1 3
[2,] 2 4
>l=matrix(TRUE,3,2)
>l
 [,1] [,2]
[1,] TRUE TRUE
[2,] TRUE TRUE
[3,] TRUE TRUE
>x[l] [1] 1 2 3 4 NA NA
```

If we assign some number some number to m[l] then the number will replace each of the values in m[l] as it happens in case of vectors. This is shown below:

```
>m[l]=4
>m[l]
[1] 4 4 4 4 4 4
```

As in mathematics, we can perform addition, subtraction, multiplication, division, modulo operation(%%), integer division(%%) and exponentiation(^) with the matrix elements by any scalar value. The operation will be done with each of the elements of the matrix. However, if we use a vector in place of a scalar for all the

Matrix Manipulations–Contd.

For the first row of the matrix, the specified operation takes place normally with the corresponding elements of the vector and that of the matrix; but for the next row of values of the matrix, the same operation takes place with the reversed values of the vector and so on. This illustrated below:

```
>m
 [,1] [,2]
[1,] 5 4
[2,] 2 5
[3,] 5 5
>v=c(4,5)
>m*v
 [,1] [,2]
[1,] 20 20
[2,] 10 20
[3,] 20 25
```

Matrix Multiplication

R supports **multiplication of two matrices** also. **The multiplication operator is %*%**. So, if A is matrix of order m x n, and B is a matrix of order n x p; then we know that the matrices are eligible for giving the product and we can perform it by simply typing A%*%B at the R prompt. The product can be assigned to a variable also which will be another matrix of order m x p , by the definition of matrix multiplication.

Relevant functions for matrix manipulations

To obtain the vector of elements on the main diagonal of a matrix M, say, we simply need to issue the command **diag(M)**.
We can also obtain the **transpose of a matrix M**, which is a matrix obtained from m by interchanging the rows and columns.
This can be achieved by simply typing **t(M)** at the R prompt and then pressing **ENTER**.
We can also obtain the **determinant value of a matrix** by simply entering **det(matrix-name)**. However, **the matrix must be a square matrix**. Sometimes, we need to extract statistics from the rows or columns of a matrix. Let f be a function that generates a number for any given vector v, If M is a matrix then, we can enter **apply(M,1,f)** to obtain the result of applying f to each row of the matrix M. The application of the **apply( )** function will generate a vector for each row of the matrix M. To obtain the similar result for each of the columns, we are to enter: **apply(M,2,f)**. We can also find out the **eigen values and eigen vectors** from given matrix by using the **eigen()** function. Above discussions have been illustrated in the figure below:

Matrix manipulations illustrated

```
>x=matrix(1:16,4)
>y=matrix(3:8,2)
>x%*%y
 [,1] [,2] [,3]
[1,] 19 29 39
[2,] 26 40 54
[3,] 33 51 69
>diag(x%*%y)
[1] 19 40 69
>t(x%*%y)
 [,1] [,2] [,3]
[1,] 19 26 33
[2,] 29 40 51
[3,] 39 54 69
>f=function(v){return(sum(v))}
>apply(x,1,f)
[1] 5 7 9
>x=matrix(1:6,3)
>apply(x,1,f)
[1] 5 7 9
>x=matrix(1:4,2)
>v=c(1,2)
>f=function(v){return(sum(v))}
>apply(x,1,f)
[1] 4 6
>x
>x=matrix(1:4,2)
>det(x)
[1] -2
>det(m) Error in determinant.matrix(x, logarithm = TRUE, ...) :
 'x' must be a square matrix
>eigen(x)
eigen() decomposition
```

Matrix manipulations illustrated–Contd.

$‘values‘
[1] 5.3722813 -0.3722813
$vectors

Finally, we shall mention two more functions frequently used with matrices. These are **rownames( )** and **colnames( )** functions. The former one is used to name the rows of a matrix; whereas, the latter one is used to name the columns. This is illustrated below:

```
>rownames(m)=c('r1','r2','r3')
>colnames(m)=c('c1','c2')
>m
c1 c2
r1 5 4
r2 2 5
r3 5 5
```

Arrays in R

We know that an array is basically a set of contiguous memory locations holding data of same datatype. The set of memory locations may be of one or more dimensions. But in R, it is a bit different. The concept of an array in R is an extension of the concept of a vector to more than two dimensions. In R, arrays are used to represent multidimensional data of a single type. R provides with an array() function for the declaration and creation of an array. The syntax of the function is as shown below:

array(data=NA, dim=length(data),dimnames=NULL)

where, **data** represents the input vector; '=NA' is not required, if data-set is replaced with data. The parameter **'dim'** represents an integer of length one or more giving the maximal subscripts in each dimension. The **'dimnames'** is either NULL or a list of names for the rows. The following codes illustrate the use of the **array()** function:

```
array(data=1:12,dim=c(2,3,2),dimnames=list(c("ONE","TWO")))
, , 1
 [,1] [,2] [,3]
ONE 1 3 5
TWO 2 4 6
```

Illustration of arrays in R

```
, , 2
 [,1] [,2] [,3]
ONE 7 9 11
TWO 8 10 12
```

If the **'dim'** is not mentioned, then the array becomes an array of one dimension. The following codes illustrate the idea.

```
>v1=c(5,7,9)
>v2=c(11,13,15)
>A=array(c(v1,v2))
>A
[1] 5 7 9 11 13 15
>A=array(c(v1,v2),dim=c(2,3,1))
>A
, , 1
 [,1] [,2] [,3]
[1,] 5 9 13
[2,] 7 11 15
```